

Floating point operations. When computers carry out addition, subtraction, multiplication, division, or other arithmetic operations on numbers represented in floating point format, the result is rounded to the nearest floating point number. These operations are called *floating point operations*. The very small error in the computed result is called (floating point) *round-off error*. In most applications, these very small errors have no practical effect. Floating point round-off errors, and methods to mitigate their effect, are studied in a field called *numerical analysis*. In this book we will not consider floating point round-off error, but you should be aware that it exists. For example, when a computer evaluates the left-hand and right-hand sides of a mathematical identity, you should not be surprised if the two numbers are not equal. They should, however, be very close.

Flop counts and complexity. So far we have seen only a few vector operations, like scalar multiplication, vector addition, and the inner product. How quickly these operations can be carried out by a computer depends very much on the computer hardware and software, and the size of the vector.

A very rough estimate of the time required to carry out some computation, such as an inner product, can be found by counting the total number of floating point operations, or FLOPs. This term is in such common use that the acronym is now written in lower case letters, as flops, and the speed with which a computer can carry out flops is expressed in Gflop/s (gigaflops per second, *i.e.*, billions of flops per second). Typical current values are in the range of 1–10 Gflop/s, but this can vary by several orders of magnitude. The actual time it takes a computer to carry out some computation depends on many other factors beyond the total number of flops required, so time estimates based on counting flops are very crude, and are not meant to be more accurate than a factor of ten or so. For this reason, gross approximations (such as ignoring a factor of 2) can be used when counting the flops required in a computation.

The *complexity* of an operation is the number of flops required to carry it out, as a function of the size or sizes of the input to the operation. Usually the complexity is highly simplified, dropping terms that are small or negligible (compared to other terms) when the sizes of the inputs are large. In theoretical computer science, the term ‘complexity’ is used in a different way, to mean the number of flops of the best method to carry out the computation, *i.e.*, the one that requires the fewest flops. In this book, we use the term complexity to mean the number of flops required by a specific method.

When I was going through the book the place that stuck me was when floating point results in round – off errors which have no practical effect, then why cant we use integers or whole number in place of float.